`scripts/tag.sh` — User Guide

Last verified: 2026-08-26 (three fail-opens closed: size-dependent evidence-staleness scan, presence-not-content evidence-pack contract, hardcoded `compileSdk` seed) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate)

Overview

Stub doc generated from the script's in-source header comment. See `scripts/tag.sh` for canonical behavior. This stub exists so the `CM-SCRIPT-DOCS-SYNC` pre-push gate (`.githooks/pre-push` Check 9) starts gating modifications to this script.

The script's own header documentation (verbatim):

```
tag.sh — Lava release-tagging tool.

Tags each app/service with `Lava-<App>-<versionName>-
<versionCode>`,
pushes every tag to all configured upstream remotes, then bumps
the
corresponding versionName/versionCode in source files and pushes
the
bump commit.

See docs/TAGGING.md for the full operator guide.
```

Usage

See the script's in-source comment block (above) for canonical usage examples.

2026-08-26 — three fail-opens closed in the release gates

Every change below has the same shape: a gate that returned a **positive verdict** in a state where it had verified nothing. Each was measured against the real script before being fixed, and each fix replaces a silent fallback with a refusal.

1. The evidence-staleness scan was weakest exactly where the evidence was stalest

Both the api-go pretag gate and (now) the Android CI-evidence gate must answer one question: *has anything outside `.lava-ci-evidence/` changed between the commit the evidence covers and `HEAD`?* The old spelling was the obvious one:

```
git diff --name-only "$base..HEAD" -- | grep -qvE '^\.lava-ci-
        evidence/'
```

Under this script's `set -Eeuo pipefail`, that pipeline is **size-dependent**. `grep -q` exits at the **first** matching path and closes the pipe; once the listing exceeds the 64 KiB pipe buffer, `git` is killed by `SIGPIPE` and exits 141, `pipefail` promotes 141 to the pipeline's status, and the enclosing `if` reads **false**. A match is thereby delivered to the caller as a **no-match**.

Measured 2026-08-26 against the real gate:

Changed files between evidence commit and HEAD	Staleness detected
1	10 of 10 runs
2000	**0 of 10 runs** — and the gate printed the positively false claim "*only `.lava-ci-evidence/` changed since*"

The more the tree had moved on, the more certainly the gate said it had not. That is the inverse of what a staleness check is for.

The fix is the helper `_scan_changed_since <base>`: the listing is captured whole, git's own exit status is checked explicitly, and the scan uses a reader that consumes to EOF. No pipe, no early-exiting consumer, no size-dependent verdict. It sets `_STALENESS_EXAMINED` (how many changed paths were actually looked at) and `_STALENESS_OFFENDERS` (the non-evidence ones), and returns **2** when git could not produce the listing at all — a failed listing is **never** reported as "nothing changed".

Both call sites now report the corpus they examined rather than asserting a bare conclusion: `examined <n> changed path(s) since; all under .lava-ci-evidence/`. A failed listing is a refusal to tag: *the freshness cannot be established, so the evidence is unverifiable.*

2. The evidence pack asserted path presence, not content

`require_evidence_for_android`'s own header states that the pack certifies three things: that `scripts/ci.sh --full` ran green, that the bluff-audit hunt ran, and that the mirror smoke test passed. The checks were `[[ -f ... ]]` and `[[ -d ... ]]`, which answer only "*does a path of the right type exist*" — something two `mkdir`s and a `touch` satisfy.

Measured 2026-08-26: a pack whose `ci.sh.json` was **zero bytes** and whose `bluff-audit/` and `mirror-smoke/` were **empty directories** reached `SP-3a evidence pack OK` and exit **0** — byte-for-byte the same verdict as a pack carrying real evidence. An empty path certifies none of the three claims. Presence is not evidence.

The fix — three content assertions:

- `_pack_json_object_ok <path>` — true only for a **non-empty regular file whose bytes parse as a JSON object**. A directory fails; a zero-byte file fails; `null`, `[]`, a bare number or string fail the type check. `jq` is required, and its absence is itself a failure with a stated reason rather than a skip.
- `_pack_dir_has_json <dir>` — true only when the directory contains at least one `*.json` that is itself a non-empty JSON object. The pack contract names `<recent>.json` *inside* these directories as the evidence, and `-d` never looks inside. `an empty directory is not evidence`.
- `real-device-verification.md` is additionally rejected when it is **zero bytes**.

Failures now name the reason per artifact, e.g. `ci.sh.json (is zero bytes — an empty file certifies nothing)`, and the summary reads `evidence pack incomplete or contentless`.

3. `ci.sh.json` must certify a green `--full` run against this commit

Content-parsing alone would still accept `{}`. The new `_require_ci_sh_json` asserts the three facts the pack header claims that file carries — **which mode ran**, **that every gate passed**, and **which commit it ran against** — making this gate the file's contract:

```
{ "mode": "--full", "all_gates_passed": true, "sha": "<40-hex>" }
```

- `.mode` must be `--full` (or `full`). *A file that does not say which mode ran certifies nothing.*
- `.all_gates_passed` must be literally `true`. Note the deliberate spelling: `jq`'s `//` operator **also fires on the boolean `false`**, so `.all_gates_passed // false` would make an **absent** field and an explicit `false` indistinguishable. The check uses an explicit `has()` test and reports `<absent>` distinctly — the same hazard and the same spelling as the Group B `gating` gate elsewhere in this script.
- `.sha` must be 40 hex characters, must resolve to a commit in this repository, and must be `HEAD` **or an ancestor of** `HEAD`. *CI evidence from an unrelated commit does not certify the commit being tagged.*
- Freshness is then checked with `_scan_changed_since` above, for the same reason the api-go gate checks it: **a CI run against a commit whose code has since changed certifies the code that was tested, not the code about to be tagged.**

This pairs with the same-day change in `scripts/ci.sh`, which now records `device_tests=ran|skipped` in its evidence directory on every exit path and refuses to report `--full` as passing when the device gate never ran.

4. `compileSdk` is derived, or the tag is refused

§6.I clause 2 requires the matrix to cover API 28, 30, 34 **and the project's current `compileSdk`**. That value is therefore this gate's own **expectation**: a wrong `compileSdk` silently redefines what "complete coverage" means.

The value was seeded `local compile_sdk=35` before the parse, and the missing-file branch had **no `else` at all**. An absent `buildSrc/src/main/kotlin/lava/conventions/AndroidCommon.kt` therefore left the gate asserting a coverage requirement no manifest supports — silently, with no warning. The decisive control, identical evidence and identical pack, differing only in whether the manifest was present:

State	Verdict
`AndroidCommon.kt` present, declares `compileSdk = 36`; attestation rows 28/30/34/35	`FATAL … matrix coverage incomplete. Missing API levels: 36 (project's compileSdk requirement)` — exit **1**
Same evidence, `AndroidCommon.kt` **absent**	`VERDICT: matrix coverage gate PASSED (compile_sdk=35 …)` — exit **0**

The worse state — no manifest at all — passed where the known state failed.

The fix derives the value or refuses. Both failure routes are refusals:

- **No file at that path** → cause reported as *checkout artifact, or the convention module moved*.
- **File present but declaring no readable `compileSdk = <n>`** → cause reported as *its shape changed*. This branch is refused too, for two reasons. A floor with one stair is not a floor — a truncated or reshaped file is the likelier drift and would have kept the same silent `35`. And the old `warn`-then-continue arm was in fact **unreachable**: under `set -Eeuo pipefail`, a no-match `grep | head | grep` assignment aborts the shell outright, so a reshaped file killed `tag.sh` with **no message at all** (verified 2026-08-26: exit 1, zero output) — non-zero, but not a diagnosis anybody can act on.

The refusal message states what was examined, what was expected, why a built-in default is not an acceptable fallback (*it asserts a coverage requirement no manifest supports, and goes stale the moment the project moves to a newer `compileSdk`*), cites the measured control above, and names both remedies: restore the file, or update this gate's parser if the convention module legitimately moved.

The block is delimited by `BEGIN/END-OF-BLOCK compileSdk derivation floor` sentinel comments so a regression harness can extract the shipped code verbatim rather than test a copy that could drift from it.

Operator consequence

A tag attempt that previously succeeded on a contentless pack, on stale evidence over a large diff, or on a checkout missing `AndroidCommon.kt`, now **fails with a named cause and a remedy**. That is the intended outcome. None of these gates gained a bypass flag, and none should: a flag that converts "*could not verify*" into "*verified*" is the class of defect all four of these changes exist to remove.

Maintenance

When this script is modified, update this document in the same commit (CM-SCRIPT-DOCS-SYNC requires it). Per §11.4.18, the documentation MUST stay in sync with the codebase — no doc may be out of sync with its script.

Cross-references

- `scripts/tag.sh` — the script itself
- `docs/helix-constitution-gates.md` — gate inventory
- HelixConstitution `Constitution.md` §11.4.18 (the mandate)
- Lava `CLAUDE.md` §6.AD (HelixConstitution Inheritance)